CS 212 Midterm Study Guide — Weeks 1–4 | Numbers are footnotes (sources) at the end

1. What is AI?[1]

- **Dartmouth workshop** (1956, John McCarthy, who coined "AI"): "every aspect of learning… can in principle be so precisely described that a machine can be made to simulate it."
- **Symbolic AI / GOFAI** ("Good Old Fashioned AI", Haugeland): explicit rules and logic on human-readable *symbols*. `IF feathers AND flies THEN bird`.
- **Modern (US 2020 law) definition:** a machine-based system that makes **predictions, recommendations or decisions**. It *perceives* input, *abstracts* it into a model (**training**), and uses the model to choose options (**inference**).
- **Categories:** Symbolic (GOFAI) | Machine learning (statistical, ANN).

2. Python Basics[2]

- No braces (**indentation** makes blocks), no `;`, **dynamically typed**, interpreted, style guide = **PEP 8**.
- Three control structures: **sequence, selection, repetition**.
- Multi-paradigm; **duck typing**; first-class functions. OOP: **composition** (has-a), **inheritance** (is-a), **encapsulation**.

Type	Syntax	Ordered	Mutable	Duplicates
list	`[1, 2]`	yes	yes	yes
tuple	`(1, 2)`	yes	**no**	yes
set	`{1, 2}`	no	yes	**no**
dict	`{"k": 1}`	insertion	yes	keys unique

Dict keys must be immutable (a tuple can be a key; a list can't).[3]

```
d = {"apples": 50}
d["apples"]            # 50     d["kiwi"] -> KeyError
d.get("kiwi", 0)       # 0 (default instead of KeyError)
unique = set([1, 2, 2, 3])        # {1, 2, 3}
name, age = ("Alice", 30)         # tuple unpacking
```

Loops: `for x in iterable:` (known items; `range(3)` gives 0,1,2) | `while condition:` (unknown count; change the condition or loop forever). `break` exits the loop; `continue` skips to next pass; loop `else` runs only if no `break`.[4]

```
for i in range(1, 4):
    if i == 2:
        continue
    print(i)           # prints 1, then 3
```

3. Python Projects[5]

- Each project gets its own **virtual environment** (`.venv`): avoids version conflicts, documents dependencies, protects system Python. Don't commit `.venv`.
- **venv/pip:** `python -m venv .venv`, activate (`.venv\Scripts\Activate.ps1` Windows, `source .venv/bin/activate` Mac), `python -m pip install pkg`, `pip freeze > requirements.txt`.
- **uv** (one fast tool; no activation needed):

Command	Does
`uv init`	new project (`pyproject.toml`, `.python-version`, `main.py`)
`uv add pkg`	install + record in `pyproject.toml` + update `uv.lock`
`uv sync`	rebuild `.venv` from `uv.lock` (cloned project)
`uv run main.py`	run in project's environment

- `pyproject.toml` = project settings and dependencies (TOML format). `uv.lock` = exact versions of everything.

4. Constraint Solving[6]

- **Declarative**: describe *what* a correct answer looks like, not *how* to find it. Part of symbolic AI; **deterministic**; answer is guaranteed to obey the rules and it can prove none exists.
- **Three inputs: variables** (unknowns), **domains** (allowed values), **constraints** (rules).
- **Results:** `sat` (solution found), `unsat` (rules contradict), `unknown` (gave up/timeout).
- Why not loops? Combinations explode (20 guests have about 2.4 quintillion seatings); solvers prune whole branches.
- Fits scheduling, gates, delivery, timetables, Sudoku. Key question: *"What are the unknowns and what are the rules?"*

5. Z3 (SMT solver)[7]

Five steps: **create variables, create solver, add constraints, check, read model.**

```
from z3 import Ints, Solver, Distinct, Abs, sat
ana, ben, cat, dev = Ints('ana ben cat dev')
s = Solver()
for f in (ana, ben, cat, dev):
    s.add(f >= 1, f <= 4)          # always bound Ints
s.add(Distinct(ana, ben, cat, dev))
s.add(Abs(ana - ben) > 1, cat == 1, ana < ben)
if s.check() == sat:               # check BEFORE model()
    m = s.model()
    print(m[ana].as_long())        # 2 (Python int)
```

- Z3 variables are **symbols**, not numbers: `Int`, `Real`, `Bool`, `Ints('a b')`. Package `z3-solver`, import `z3`.
- `==` is a constraint; `=` is assignment and destroys your symbol. Operators are overloaded to build rules.
- Use `And Or Not Implies If Distinct Sum Abs`, **not** Python `and/or/not/if`. `If(c, 1, 0)` counts.
- `Implies(a, b)` only restricts when `a` is true.
- Solver: `add`, `check`, `model`, `push`/`pop` (what-if), `reset`, `set("timeout", ms)`. `unsat_core()` (with tracking flags) shows conflicting rules; `Optimize` has `minimize`/`maximize`.
- Z3 gives *a* solution, not *the* only one.

6. Machine Learning Basics[8]

- ML = models trained on data sets to **predict** and **classify**. A **model** = algorithm + what it learned from training data.
- **Feature**: one piece of input info. **Label**: target output. **Training data** teaches; **test data** (unseen) evaluates.
- **Supervised** (labeled data; spam filter) | **Unsupervised** (unlabeled, finds structure; k-means customer groups) | **Reinforcement** (agent learns by trial, rewards/penalties).

Model	Idea
Naive Bayes	Bayes' rule, assumes features independent; fast
Bayesian network	directed graph; models feature dependencies
k-NN	classify by majority vote of *k* nearest feature vectors; no real training step, just stores data
Linear regression	`y = mx + b`; predicts a continuous value
Logistic regression	sigmoid squeezes to 0–1; binary classification (multinomial = unordered 3+, ordinal = ordered 3+)
Decision tree	flowchart: decision nodes test features, leaves are outcomes
Random forest	many trees on random samples; majority vote (or average); reduces overfitting
ANN	weighted connected neurons; weights adjusted in training

7. Bayes' Rule[9]

- **Probability** = happens / total (die 6: 1/6). **Odds** = happens : doesn't (1:5). Odds o to probability: $o / (1 + o)$.
- **Prior odds**: before new evidence (rain in Eugene 146:219).
- **Likelihood ratio**: P(observation | event) / P(observation | no event) (morning clouds: 7).
- **Posterior odds = prior odds × likelihood ratio** = 146:219 × 7 = 1022:219 (82.4%).
- **Naive Bayes sentiment:** label posts, count word frequencies per class, then `likelihood = (negFreq/negTotal) / (posFreq/posTotal)` (e.g. fine: 10/500 ÷ 50/1000 = 0.40).
- **Inference:** prior × likelihood$_1$ × … × likelihood$_n$, compare to a **decision threshold** found from training data.

8. Data Sets[10]

- **Garbage in, garbage out. Class** = category; **label** = its integer id (0, 1, 2…). **Feature vector** = the numbers describing one sample.
- **Feature types:** continuous (float), interval (integer, equal steps), **ordinal** (order, unequal gaps), **categorical** (codes, no order; use **one-hot encoding**: male = [1,0], female = [0,1]).

- Include only features that help separate classes. **Curse of dimensionality:** needed data grows about 10^d with d features.
- **Interpolation** (inside training range) works; **extrapolation** fails. Train, test and real data must share one **parent distribution**.
- Match real **class priors**, except rare classes (may balance training set). Include **confusers** (hard negatives, e.g. wolves in "not dog").
- **Scaling:** *mean centering* (subtract mean → mean 0); *standardizing* (subtract mean, divide by σ → mean 0, SD 1). **Missing values:** impute with the feature's mean.
- **Split:** training (~90%) fits parameters; **validation** (~5%) tunes hyperparameters; **test** (~5%) used **once** at the end. Split by class or randomly.
- **k-fold cross validation** (small data sets): k folds, train k times, each fold tests once, average results.
- Inspect data: mislabels, outliers (box plot whiskers), mean/SD/median; beware models learning confounders (sunny vs. cloudy tanks).

9. scikit-learn Text Pipeline[11]

- **Bag of words:** each word gets a column; cell = count. High-dimensional and **sparse** (mostly zeros).
- **`CountVectorizer`** builds vocabulary + counts. **tf** = counts scaled by doc length; **tf-idf** = also down-weights words common to many documents.
- **`fit`/`fit_transform` on training data only; `transform` on new data** (reuse the learned vocabulary).
- **`Pipeline`** chains steps; `step__param` names a setting.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer,
        TfidfTransformer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split,
        GridSearchCV
tr_x, te_x, tr_y, te_y = train_test_split(msgs, labels,
                         test_size=0.25, random_state=42)
pipe = Pipeline([("vect", CountVectorizer()),
                 ("tfidf", TfidfTransformer()),
                 ("clf", MultinomialNB())])
pipe.fit(tr_x, tr_y)              # train
pred = pipe.predict(te_x)         # inference
grid = {"vect__ngram_range": [(1, 1), (1, 2)],
        "clf__alpha": [0.1, 1.0]}
gs = GridSearchCV(pipe, grid, cv=5, scoring="f1")
gs.fit(tr_x, tr_y)                # best_params_, best_score_
```

10. Classifier Metrics[12]

Pick a **positive** class (spam = 1). **Confusion matrix** (rows = true, cols = predicted); sklearn 2-class order is `[[TN, FP], [FN, TP]]`.

	Pred. spam	**Pred. ham**
Is spam	TP	FN
Is ham	FP	TN

Metric	Formula	Asks
Accuracy	(TP+TN) / all	% right overall
Precision	TP / (TP+FP)	when it says spam, how often right?
Recall	TP / (TP+FN)	of real spam, how much found?
F1	2PR / (P+R)	high only if both high

- **Accuracy misleads with a rare class:** "always ham" scores 1203/1394 = 86.3% but recall 0. Spam model: TP 178, FP 6, FN 13, TN 1197 → precision 178/184 = 0.967, recall 178/191 = 0.932.
- Precision and recall trade off; choose by cost of each mistake (spam filter: false positives worse → precision; emergency alerts: recall).
- **Multiclass:** each class takes a turn as positive. Row total = **support**; diagonal / row = recall; diagonal / column = precision. **Macro avg** = plain average of classes; **weighted avg** = by support.
- **Hyperparameters** are set *before* training (`alpha` smoothing, `ngram_range`, `class_weight="balanced"`). Tune with `GridSearchCV`; for a rare class use `scoring="f1"` (or `"f1_macro"`), not accuracy.

```
from sklearn.metrics import (accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix,
        classification_report)
tn, fp, fn, tp = confusion_matrix(te_y, pred).ravel()
precision_score(te_y, pred, zero_division=0)
print(classification_report(te_y, pred))
```

1. Overview of AI: Dartmouth, McCarthy, GOFAI, the 2020 definition, categories.
2. Intro to Python: Python distinctives, paradigms, OOP concepts.
3. Python Collection Types: lists, dictionaries, `get()`, sets, tuples.
4. Python Loops: `for`, `while`, `break`, `continue`, `else`.
5. Python Projects: `.venv`, pip, `requirements.txt`, uv, `pyproject.toml`.
6. Constraint Solvers: variables, domains, constraints, sat/unsat, real-world uses.
7. The Z3 Constraint Solver: five steps, Z3 functions, solver methods, pitfalls, cheat sheet.
8. Machine Learning: terminology, types of ML, supervised models.
9. Machine Learning: Bayes' Rule: odds, prior, likelihood ratio, posterior, sentiment example.
10. Data Set Concepts and Preparation: features, dataset quality, scaling, splitting, k-fold, inspection.
11. Working with Text Data (week 3 scikit-learn tutorial): bag of words, tf-idf, pipeline, grid search.
12. Classifier Metrics: confusion matrix, accuracy/precision/recall/F1, multiclass reports, tuning.